

CHOWLY

A dining platform for The Golden Gate, Lagos

Deborah Akinbola

Repository <https://github.com/DebbyA007/CHOWLY>

Deployed <https://chowly-theta.vercel.app>

This document answers the three deliverables of the CHOWLY build assignment in the order the brief lists them. It is written to be read without any other document open.

Contents

Deliverable 1: The git repository	4
Deliverable 2: The URL of the deployed application	4
Deliverable 3: The document	4
1. How I built it	4
The stack, and why I chose it	4
How I organised it	5
The menu, which I wrote myself	5
The staff, and how I made the app decide who gets recorded	6
The photographs	7
The data model as I finally implemented it	7
How I deployed it	8
2. How I used AI	8
What I used, and how I worked with it	8
What I asked for	9
What I accepted	9
What I rejected	9
What I had to correct myself	10
3. The specific behaviour of the application, from menu to payment	10
Menu browsing	11
Order placement	11
The wait, and why it is the centre of the screen	11
Complaint and rating	12
Order assignment	12
Payment	12
Something to do while you wait, which is beyond the requirements	13
Cancelling an order, which is beyond the requirements	13
The two roles, and moving between them	13
Real storage	14
Two more things I built beyond the requirements	14
4. How to use it: a walkthrough a stranger can follow	14
A recorded walkthrough	15

Where each requirement is answered 15

Deliverable 1: The git repository

My code is at <https://github.com/DebbyA007/CHOWLY>. It is public, so facilitators need no invitation to read it.

I did not squash anything or rewrite any history, so the commits are the work as it actually happened, including the ones where I broke something and fixed it. Three worth opening:

- "feat: replace the menu with the full card, and four model deltas". I replaced my first eleven placeholder dishes with the ninety two I wrote for the restaurant, and four changes to the data model came with them.
- "fix: placing an order was returning 400 on production". I added a field the client needed for itself to the request body it posts, and my own validation refused every order. Described under how I used AI.
- "fix: order placement failed permanently on production". I derived the order number from a count of orders, which stops working the moment one is deleted.

The repository also carries the assignment brief and the engineered model I built from, at docs/assignment/, so my work can be checked against what it is marked on without leaving the repository.

Deliverable 2: The URL of the deployed application

<https://chowly-theta.vercel.app>

It opens on the front door, with nothing to install and nothing to sign in to, and both roles reachable from that first screen. Adding `?table=12` to the URL is what a QR code on a physical table would carry; without it the door asks for the table number.

Deliverable 3: The document

The four required parts follow in the order the brief lists them.

1. How I built it

The stack, and why I chose it

The brief says I am marked on the solution rather than the stack, so I chose tools that would keep me honest rather than tools that would be interesting.

I chose **Next.js with TypeScript** because one project serves both the pages and the API, which means the type the server computes is the same type the client renders, and a mismatch is a compile error rather than a bug a guest finds. I chose **PostgreSQL on Neon with Prisma** because migrations are then files in my repository that a reader can inspect, and because Neon provisions through Vercel, so the database and the deployment were one setup step. I chose **Zod** to validate every request, and it earned its place more than once: because my routes reject any field they do not expect, a bug of mine that would otherwise have corrupted an order arrived instead as a clear refusal naming the field.

The rest follows the same logic. **Tailwind** so my design tokens live in one stylesheet and nothing else defines a colour. **SWR** so a polling screen keeps its previous data while it revalidates and nothing blinks. **anime.js** for motion that cleans itself up. **Vercel** because it deploys on push.

How I organised it

The rule I held to is that any number a guest sees is computed in exactly one place, on the server. The promised wait is computed in one file and nowhere else; the money in another. The client may draw a countdown, but it may not decide one. That single rule is why a guest cannot post a price, why a refresh never moves the clock, and why I could unit test the parts that matter without a browser.

- **app/** holds the routes, with the pages under `app/(guest)` and `app/waiter` and the API under `app/api`. I kept every route handler to a few lines: parse, authorise, compute, respond.
- **lib/** holds everything that decides something. The wait time, the money, the session token, the cart, the greeting, the cancel window and the order number are each one small file with tests beside it. Nothing in `lib` imports a React component, so none of it needs a browser to test.
- **components/night/** holds the screens, named for what they show.
- **prisma/** holds the schema, the migrations and the seed.

The menu, which I wrote myself

I wrote all ninety two dishes for this restaurant myself: the dishes, their descriptions, the prices and the seven sections they sit in. It is original work, not drawn from any existing restaurant, and I priced it at Lagos fine-dining level so the totals a marker sees are plausible. The brief asks for a menu loaded into the database by me, and this is that menu.

I gave the card two levels because a printed menu has two levels: a heading you scan for, and sub-headings under it.

Heading	Sub-sections	Dishes
Breakfast	Food, Drinks	15
Lunch	Starters, Mains, Drinks	19
Dinner	Starters, Mains	13
Finger foods	one section	10
Desserts	one section	6
Drinks	Cocktails, Wines, Soft, Hot	27
Tasting menu	one section	2

Seven headings, fourteen sub-sections. Every dish carries a name, a description, a price and a preparation time, which is what the brief requires. Three of the bottles I priced from a floor rather than a fixed figure, the way a real cellar list does, so they show "from N75,000", say to ask your waiter, and cannot be added to an order at all. I would rather refuse an order than put a number on a bill that nobody agreed.

I set the preparation times myself, because I had not written any into the menu. My card had three columns, dish, description and price, and the brief requires a time on every item with the promised wait computed from it. So I went through all ninety two by hand, reading what each description says is done to the dish, and gave it a number. Thirty two distinct values from one minute to ninety:

Section	Minutes	Dishes
Breakfast, food	12 to 22	8
Breakfast, drinks	3 to 5	7
Lunch, starters	11 to 16	5
Lunch, mains	22 to 30	8
Lunch, drinks	1 to 5	6
Dinner, starters	10 to 22	5
Dinner, mains	26 to 45	8
Finger foods	9 to 16	10
Desserts	8 to 15	6
Drinks, cocktails	5 to 6	5
Drinks, wines	3 to 4	6
Drinks, soft	1 to 5	9
Drinks, hot	3 to 5	7
Tasting menu	5 and 90	2

I chose the two ends deliberately. Premium Still Water takes one minute, so a marker can order it alone and watch the order run late without waiting for a kitchen. The chef's tasting menu takes ninety, which is also the ceiling I put on the wait calculation.

The staff, and how I made the app decide who gets recorded

I seeded three waiters, three chefs and two bartenders for the waiter to pick from. Then I noticed a problem with the model I had been given: it assumes every order has a chef and a bartender, and a glass of water has neither.

So I gave every dish a station, and let the order work out for itself who made it.

Station	Dishes	What it means
Kitchen	51	Something is cooked, so the order needs a chef
Bar	38	Something is mixed, so the order needs a bartender
Neither	3	Still and sparkling water are poured, so neither is needed

A dish and a cocktail asks the waiter for all three people. A round of cocktails asks for two. An order of still water asks for a waiter and nobody else, and tells the waiter why. The receipt then

credits only the people who actually made it, instead of putting a chef's name against work nobody did.

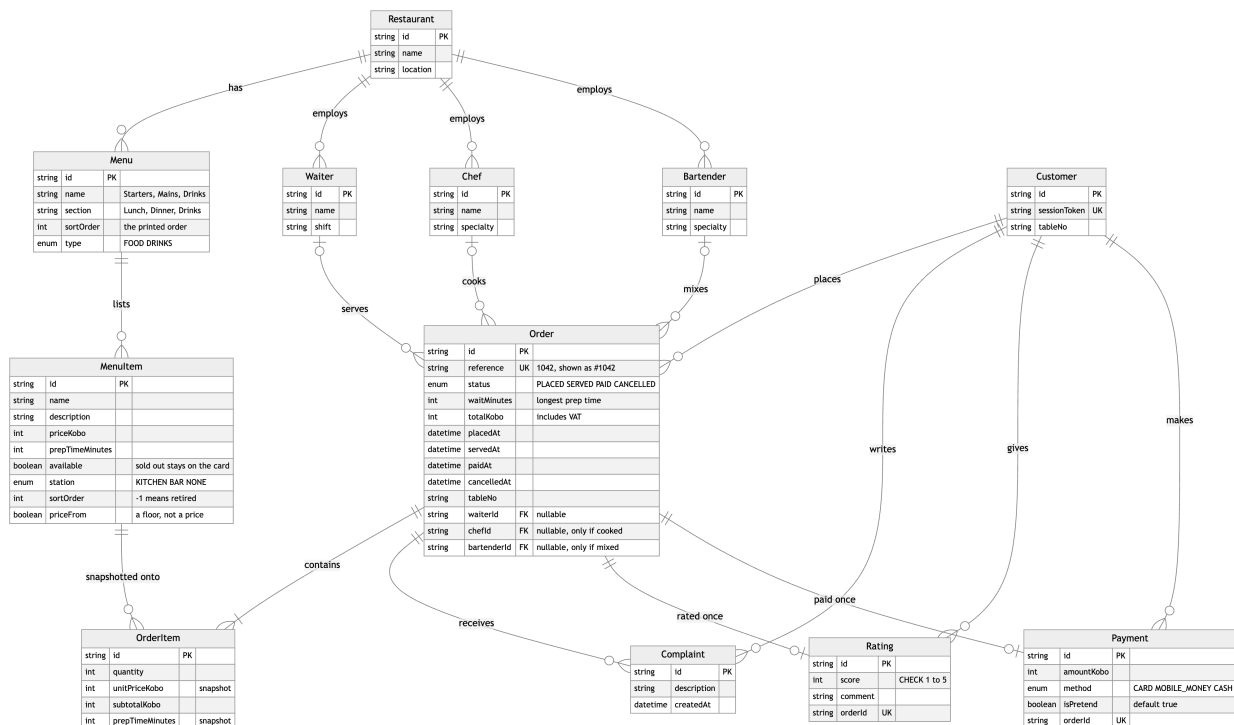
The photographs

Every one of the ninety two dishes has a photograph, and I served them all from my own repository because my Content Security Policy allows images from this origin only. I held myself to permissive licences throughout: 31 CC0, 49 CC BY 2.0, 5 CC BY 4.0, 3 CC BY 3.0 and 4 public domain, with no share-alike image anywhere on the card. Each one's source, author and licence is in docs/PHOTOGRAPHY.md.

Thirty two of them show a similar dish rather than the exact one, and I recorded that against each in the same file rather than letting the set look better than it is. Akara frying in a pan stands for the Nigerian Breakfast Platter; poached eggs stand for Moi Moi and Poached Eggs. One CSS treatment sits over all of them so they read as one set.

The data model as I finally implemented it

I implemented all twelve entities of my engineered model, with every foreign key it lists, including the customer key on Complaint, Rating and Payment. I then departed from it in fifteen deliberate ways, each marked DELTA in the schema beside the field it changes. The brief asks me to make a change the build forces and say why, so here they are, grouped by what forced them.



The data model as implemented. Every entity of the engineered model is present; the fields added or changed by the fifteen deltas are visible on Menu, Menuitem and Order.

Five because the model could not express money or time correctly. I added the preparation time the brief requires and compute the promise from it. I made the wait an integer, because the model stored a string like "25 mins" and I cannot compare that to a clock. I made the three timestamps real timestamps, because the model split date and time into separate columns and I could not sort or compare them. I put every amount in integer kobo so no floating point value goes

anywhere near a total. And I snapshot the price and the preparation time onto each order line at the moment it is placed, so editing my menu never rewrites an order somebody already paid.

Four because the model asserted things that are not true of a restaurant. The three staff keys are nullable, because a guest submits with nobody attached and the waiter records them afterwards; had I left them NOT NULL, requirement 3 would have been impossible. I added the station, described above. I added a flag for a price that is a floor rather than a price. And I gave the menu a section, a sort order and a per-dish order, because a printed card has two levels and an order that is not alphabetical, and one flat name could carry neither. A dish I retire keeps a sort order of -1: off the card, still in the database, so an order placed months ago still reads back in full.

Two because the model had no way to record a state the story needs. Late is not a status. I derive it at read time from the placed time and the promised wait and never store it, which means there is no row anyone can hand-set to make the complaint appear. And I added CANCELLED with a timestamp, because the model's statuses only ran forward and a guest who ordered by mistake had nothing to do but wait.

Four because the model left integrity to the application. I made the payment unique per order and wrote it in one transaction with the status change, so a double-tapped button records once. I defaulted the pretend flag to true so a pretend payment is pretend in the data and not only in the wording. I made the rating unique per order and added a check constraint by hand for the one-to-five range, because Prisma cannot express it: Zod validates first, the database is the last line. And I made the session token unique, since there are no logins and that token is the identity.

How I deployed it

Vercel deploys my default branch on every push. Three choices are worth stating.

- I install with `npm ci` and no install scripts, so nothing a package ships runs on the build machine.
- That also blocks Prisma's own code generation, so my build command generates the client and then builds. It does not run migrations. A deploy that silently changes the database schema is a worse problem than the one it solves, so I apply migrations deliberately instead.
- No secret of mine is prefixed `NEXT_PUBLIC_`, so nothing from my environment reaches the browser bundle.

2. How I used AI

What I used, and how I worked with it

I used Claude as a pair programmer, inside Claude Code, the terminal client that can read and write files in my repository and run commands. I used it for the whole build rather than for a part of it, and I kept a record of what I asked, took, threw away and had to fix as the work happened rather than writing it up afterwards.

The way I worked with it mattered more than the tool. I wrote my constraints down first, committed them at `CLAUDE.md`, and made them non-negotiable: money is integer kobo, every route validates strictly, the customer identity is a signed `httpOnly` cookie and never `localStorage`, ownership is checked inside the query on every route that touches an order, no flashing or screen shake in any animation. Then I asked for whole features rather than snippets, and treated everything that came back as a proposal with claims attached that I had to check.

What I asked for

Whole pieces of the product, in my own terms. Build the waiter's ticket rail. Replace my eleven placeholder dishes with my ninety two and tell me what that forces in the data model. Let a guest cancel their own order inside a window, computed on the server, and tell me why the window should be a fraction of the promise rather than a fixed number of minutes.

I also used it as something to argue with. The station idea came out of me asking why a bottle of water needs a bartender, and the answer changed the schema.

What I accepted

- **The shape of the codebase.** One computation per file, thin route handlers, screens named for what they show. I took it early and it held for the whole build, which is the strongest thing I can say about it.
- **Deriving lateness instead of storing it.** I took this immediately, because it removes a column that could disagree with the clock, and it is the reason my complaint can only appear when an order really is late.
- **The security posture.** Never trusting a price from the client, making payment idempotent with a unique constraint and a transaction, rate limiting per session in the database, and checking ownership inside the query rather than after it.
- **Most of the copy, after correcting it.** The rule I set, that an error says what went wrong and how to fix it and never apologises, produced better sentences than my own first drafts.

What I rejected

- **Three complete art directions, after building each one properly.** The first was competent and forgettable, a card grid with nothing anyone would remember an hour later. The second was well made and wrong for the job: too heavy, with decoration standing between the guest and the task. The third is the one I shipped. I kept all three in the repository at /directions, because a design decision with its rejected options attached is worth more than one simply asserted.
- **Storing a delayed flag on the order.** Two sources of truth for one fact, and the stored one would be the stale one.
- **Storing a total the client sends, and later a subtotal beside the total.** I refused both for the same reason: a figure stored twice can disagree with itself.
- **A cancel button on the waiter's side.** It was the obvious companion to the guest's cancel and a real restaurant would have one. But my waiter side opens with one tap and no login, so that button would be authorised by nothing, and unlike marking an order served, a stranger using it could not be undone by the table.
- **Thirteen category chips in one scroller** for the new menu. That is a list of tabs, not a menu, so I put the seven headings on the strip and the sub-headings under them.
- **An "In the kitchen" step in the order tracker.** It was invented from nothing my database records, so I took it out.
- **Roughly two thirds of the photographs it found for me,** because they were pictures of the wrong thing.

What I had to correct myself

This is the part I would want to read, so it is the part I have written most carefully.

- **I brought down my own live app by letting the two ends of it disagree.** I added a field the client needed for itself to the body it posts, and my own strict validation refused every order on the deployed link. What makes it worth recording is that all four of my checks stayed green throughout, because none of them compared the body my client sends with the schema my server accepts. I fixed it by separating those two things properly and adding a test that asserts the wire shape from both sides, which is the check that had been missing all along.
- **I brought it down a second time by deriving an order number from a count.** The reference was 1001 plus the number of orders, which is only ever correct if no order is deleted, and I had deleted some test orders. The count then landed on a number still in use, and because a failed insert does not change a count, all five of my retries recomputed the same number and failed identically. Every order after that failed. I now take the next number above the highest in use and increment on each retry, which is what makes it a retry. Both outages taught me the same thing: my checks measured everything except the thing that was wrong.
- **I searched for the photographs in the wrong language.** I collected over two hundred candidates using my English dish names and five dishes still came back with nothing usable. Then I searched the Nigerian names instead, and one pass finished the card: moi moi steamed in its leaves, egusi beside a ball of swallow, akara straight out of the pan. "Bean pudding" and "melon seed soup" returned nothing; "moi moi" and "egusi" returned the dish at once. The fix was to my instruction, not to any code.
- **I let it write animation code from memory of an older version of the library.** anime.js v4 renamed several things that v3 code still accepts silently, so the animations ran and did nothing. I caught it by watching the screen rather than reading the code, and corrected it against the installed version's own types.
- **I wrote a security flag that failed the wrong way.** My staff check was on unless a flag was exactly false, which locked the waiter side of my deployment where I had never set the variable at all. It is now on only when the flag is exactly true, so an unset variable leaves the app usable, which is what the brief requires.
- **I let it tell a guest things the app does not know.** The order screen said "it is with the chef now" for a glass of water, and a failure message blamed a busy kitchen for what was really a failed database write. Both sound plausible, which is why they survived. Those sentences now name the kitchen, the bar or the waiter based on what is actually on the order, and the failure says only that the order was not saved and nothing has been charged.

The pattern across all of it is that AI was reliable at structure and at following a constraint I had written down, and unreliable about anything outside the code: what a browser really does at a real size, what a photograph is actually of, what a plural should be once the data changed underneath it. Everything I checked by running it and looking held up. Everything I checked by reading the code did not. That is the habit I am taking from this project.

3. The specific behaviour of the application, from menu to payment

The five features the brief names come first, each under its own heading, then what I built beyond them.

Menu browsing

A guest opens the link at their table. The front door shows the room, the restaurant's name and address, "I'm a guest" and "I'm a waiter", and either the table number the link carried or a field asking for it.

Tapping "I'm a guest" opens the card. Under the restaurant's name is a greeting set by the hour on the guest's own device, changing at five, at noon and at five again. I read the hour from their phone rather than my server, because a guest in Lagos should not be told good evening because a machine in Frankfurt thinks so, and I used no honorific, because with no login I know nothing about who is holding the phone.

The card is the printed menu: the seven headings along the top, and a lighter second row of sub-headings under any heading that has them, so Lunch offers Starters, Mains and Drinks. Tapping either filters in place. Each row shows a round photograph, the dish, its description, its price and the kitchen's minutes. Where a description runs long I clip it and offer to show the rest, because my tasting menu lists eight courses and I did not want one dish pushing the rest off the screen.

A dish that has run out stays on the card, greyed, with "Sold out" where the add control was, so a guest can see the restaurant has it. The three bottles priced from a floor say "Ask your waiter" in the same place. The card refreshes on its own and when the tab regains focus, so a dish the kitchen takes off greys without anyone reloading.

Order placement

Tapping the circle on a row adds the dish and turns the control into a stepper. The cart bar at the bottom never disappears, so a guest always knows what they have; it shows the count and the running total, and the figures tick rather than jump.

"View order" opens the order sheet, with the lines, the subtotal, VAT and the total, and the table number. "Place order" sends the dish ids, the quantities and the table, and nothing else.

What my server does next is the part I care about. It reads the price and the preparation time of every dish from the database, snapshots them onto the order, adds up the lines, adds VAT and works out the promised wait as the longest preparation time in the order. A request carrying a price is rejected outright. A request carrying a dish that sold out while it sat on the screen is refused by name, and the guest's screen takes it off the order and says what changed.

The Order tab opens straight away with a provisional order drawn from the same formula the server uses, so nobody watches a spinner, and the real order replaces it a moment later.

The wait, and why it is the centre of the screen

The story the brief describes is really about waiting, so I made the wait the screen rather than a line of text on it. A ring empties as the promised minutes are used, recomputed every second from the time the order was placed, so a refresh lands exactly where the clock is and nothing can drift. Above it sits a vessel that matches what was ordered: a pot for anything cooked, a glass being poured for a drinks-only order.

When the promise is spent the screen crosses slowly from ochre to a late red, over about two minutes, never a snap and never a flash. The ring closes and the numbers count up instead of down.

Complaint and rating

I made the complaint something a guest earns rather than something always sitting there. "Report a problem" appears only once the order really is late, which is what the brief describes, and my server enforces the same rule, so a hand-made request cannot complain about an order that is on time. The guest writes a line and sends it, and it is stored against both the order and the customer; the waiter sees the count against that table within seconds.

"Rate your order" takes one to five and an optional comment. One rating per order, enforced by a unique constraint, so rating again changes the existing one rather than adding a second. Both the complaint and the rating appear on the waiter's copy of the order.

Order assignment

The waiter side opens with one tap and no prompt. The rail refreshes every few seconds and keeps what is on screen while it does, so nothing blinks. Each order is a card with its table and number, its items and total, and a status with its own time: just placed, in the kitchen with the time left, ready in the last minute, late with the minutes over in red, or served. Filters narrow the list.

Opening an order shows when it was placed, its clock, its lines, and any complaint or rating from the table in full. Then the staff, and this is where the station earns its place: the screen asks only for the people the order actually needs. My endpoint works the same answer out from the order's own lines, so it refuses a missing chef for a cooked order and equally refuses a chef sent for an order with nothing from the kitchen.

"Mark as served" records the staff and the time. The screen shows it at once and puts the order back if my server refuses. Only a placed order can be served; serving twice is refused rather than quietly rewritten. The guest's screen picks the change up within seconds and with no reload: the ring reads Served, the vessel becomes the plated dish or the full glass, and the screen leads with the way to settle.

Payment

The Pay tab shows the bill with the lines, the subtotal, VAT and the total, and three ways to pay. The button reads "Pay N4,838 (pretend)", and the word pretend is on the button and on the receipt, because the brief asks for a payment that is recorded and clearly labelled as pretend.

I write the payment inside one transaction that inserts the row and moves the order to paid together, and the payment is unique per order, so a double-tapped button records once rather than twice. The pretend flag defaults to true in my schema, so it is pretend in the data and not only in the wording.

Then the receipt prints, with the perforation, the lines, the total, a stamp that lands once, a torn foot, and one credit line naming the people who actually made the order. For an order of still water it reads "Served by Ada O." and names nobody else, which is the whole point of the station.

"Save the receipt" draws it again as a picture and hands it to the phone. Where the phone will carry a picture in its share sheet, that is how it reaches the photo library; where it will not, the same button downloads it instead. The line under the button says which is about to happen.

Something to do while you wait, which is beyond the requirements

The wait is the thing this app is about, so I gave a guest something to do inside it. Once an order is placed there is one quiet control on the order screen, "Play while you wait", and it opens a small panel over that screen with two games in it. Tic tac toe, alone against the app or two people passing the phone across the table, and a quiz of forty eight questions on Lagos and Nigeria, food and cooking, and general knowledge, again alone or two people taking turns. Nothing repeats within a sitting, and the right answer is shown after every question.

It is offered rather than pushed. One control, easy to ignore, gone the moment there is nothing left to wait for, and it closes on a single tap with nothing asked.

Two rules shaped the rest of it. **The countdown never goes behind the game.** The panel is capped against the ring rather than at a share of the screen, so it can never rise above it on any phone, and there is no dimming over it. If the order is served, runs late or is cancelled while someone is playing, a band appears over the game saying so with a way straight back. The wait is the subject; a game is not allowed to hide it.

And nothing is stored. A refresh closes the game and the next one starts clean. That is a decision rather than something I did not get to: a half-finished board restored beside an order that has since been served is worse than a lost game of tic tac toe, and the countdown underneath survives a refresh exactly as it always did, which is the state that actually matters. The games touch no table, no route and no request; they are entirely in the browser.

Cancelling an order, which is beyond the requirements

I added this because the model I was given only ran forward, and a guest who ordered by mistake had nothing to do but wait for food they did not want.

A guest can withdraw their own order while it is still placed and no more than a quarter of the promised wait has passed. I made the window a fraction of the promise rather than a fixed number of minutes so that it stays proportionate: a glass of water gives fifteen seconds, my tasting menu gives twenty two and a half minutes. After that the kitchen has started and cancelling would throw away food.

My server computes that window on every request. The button and its countdown are presentation, so a tap that leaves the screen inside the window and arrives outside it is refused with a sentence saying so. The order id comes from the path and the customer from the signed cookie; I never read either from the request body. When the window closes the button goes away where it stands and a sentence takes its place, without anything below it moving.

The two roles, and moving between them

The front door offers "I'm a guest" and "I'm a waiter", one tap each. On every screen after that there is a switch at the top right showing which view you are in, and one tap moves to the other without reloading the page. It is a view switch and nothing more, because there are no logins anywhere, which is what the brief allows.

What a guest can do is still bounded, and I was careful about this. Their identity is an opaque token in a signed, httpOnly cookie, never in localStorage where injected script could read it. Every read, complaint, rating, cancellation and payment checks ownership inside the database query, so one table cannot read or pay another's order. Order creation, complaints and ratings are rate limited per

session.

Real storage

Everything is in PostgreSQL: the restaurant, the menu, the dishes, the staff, the customers, the orders and their lines, the complaints, the ratings and the payments. Refreshing any screen changes nothing a guest can see. The countdown in particular is computed from the stored time the order was placed rather than from when the page loaded, so a refresh puts the ring exactly where it was.

Two more things I built beyond the requirements

- **The 86 board.** The waiter's Menu tab lists every dish with a switch. Taking one off removes it from the guests' card within the minute and refuses by name any order still carrying it.
- **The table board.** The waiter's Tables tab is the floor by table, with every order of the last twelve hours, what each table still owes, and the night's outstanding total.

4. How to use it: a walkthrough a stranger can follow

You need a phone or a browser and the link. Nothing to install, nothing to sign in to, about five minutes.

1. Open <https://chowly-theta.vercel.app/?table=12>. The dining room, the name, and "You're at table 12". Without `?table=12` the door asks for the number instead.
2. Tap "I'm a guest". The card opens on Breakfast, with the greeting for whatever hour it is where you are. The chips along the top are the seven headings; under a heading that has them, a lighter second row carries the sub-headings.
3. Tap Drinks, then Wines. The three bottles priced "from N75,000" have no add control and say to ask your waiter, because there is no price yet to put on a bill.
4. Tap Drinks, then Soft, and add one Premium Still Water and nothing else. This is the one thing to get right if you are short of time: the promise is the longest preparation time in the order, water takes one minute, so it runs late in a minute. That is the only way to see the complaint, which appears once an order is late and not before. A jollof rice would keep you waiting twenty two minutes for the same thing.
5. Tap "View order", check the table number, and tap "Place order". The Order tab opens at once and the order number lands a moment later.
6. Watch the glass and the ring. Under them is "Cancel this order" with the time you have left beside it: fifteen seconds for water. Watch it reach zero and the button goes away where it stands, with a sentence in its place and nothing below it moving. If you would rather use it, place a second order and tap it inside the window.
7. Wait for the promise to run out. The ring closes, everything crosses slowly to red, and "Report a problem" and "Rate your order" appear. Tap Report, write a line, send it. Then tap Rate, pick a low number, and send.
8. Switch to the waiter with the Waiter half of the switch at the top right. No PIN, no prompt. Your table is in the list, marked late, with the count of your report.

9. Tap the pill, "Who's serving?", and pick a name. Tap the card. Your report is there under "From the table". Notice what the screen asks for: an order of water asks for a waiter and nobody else, and says why. Tap "Mark as served".
10. Switch back to Guest at the top right, then the Order tab. Within seconds and with no reload it reads Served. Tap Pay.
11. Pick a method and tap "Pay ... (pretend)". The receipt prints, with the stamp, the torn foot, and a credit line naming only the waiter, because nothing on that order was cooked or mixed. Tap it twice if you like: the record is one payment.
12. Two more worth a minute. On the waiter side, the Menu tab is the 86 board: switch a dish to "Sold out", then look at the guests' card and find it greyed. And turn on Reduce Motion in your system settings and reload: every entrance becomes a fade, and the late colour change still happens, slower, because it is a colour and not a movement.

A recorded walkthrough

I recorded the same pass end to end at docs/media/walkthrough.mp4 in the repository: three minutes forty five, one continuous take on the deployed app, no cuts.

Where each requirement is answered

The brief asks for	Where it is
1. The menu, with a name, a price and a preparation time on every item	Ninety two dishes I wrote and priced, seeded into PostgreSQL with all three fields. The menu, which I wrote myself
2. Placing an order, with the order details and the waiting time shown	Order placement. The wait is computed on my server and shown as the ring
3. Assigning the order: a waiter records the chef and the bartender and marks it served	Order assignment. I seeded the staff, and the app works out which of them the order needs
4. Complaint and rating, both stored against the order	Complaint and rating. Both are rows against the order and the customer
5. Payment, recorded, marking the order paid, clearly labelled pretend	Payment. One transaction, a unique constraint, pretend in the data
6. The two roles, with a simple switch and no logins	The two roles. The front door, and a switch on every screen after it
7. Real storage that survives a refresh	Real storage. PostgreSQL on Neon; the clock comes from the stored timestamp
8. A live link anybody can use	https://chowly-theta.vercel.app

Beyond them I built the cancel window, the two games to play while an order is on its way, the 86 board, the table board, the receipt as a saveable picture, the staff derived from the stations, the role switch, and the design, which is one art direction I chose over three I built and rejected.